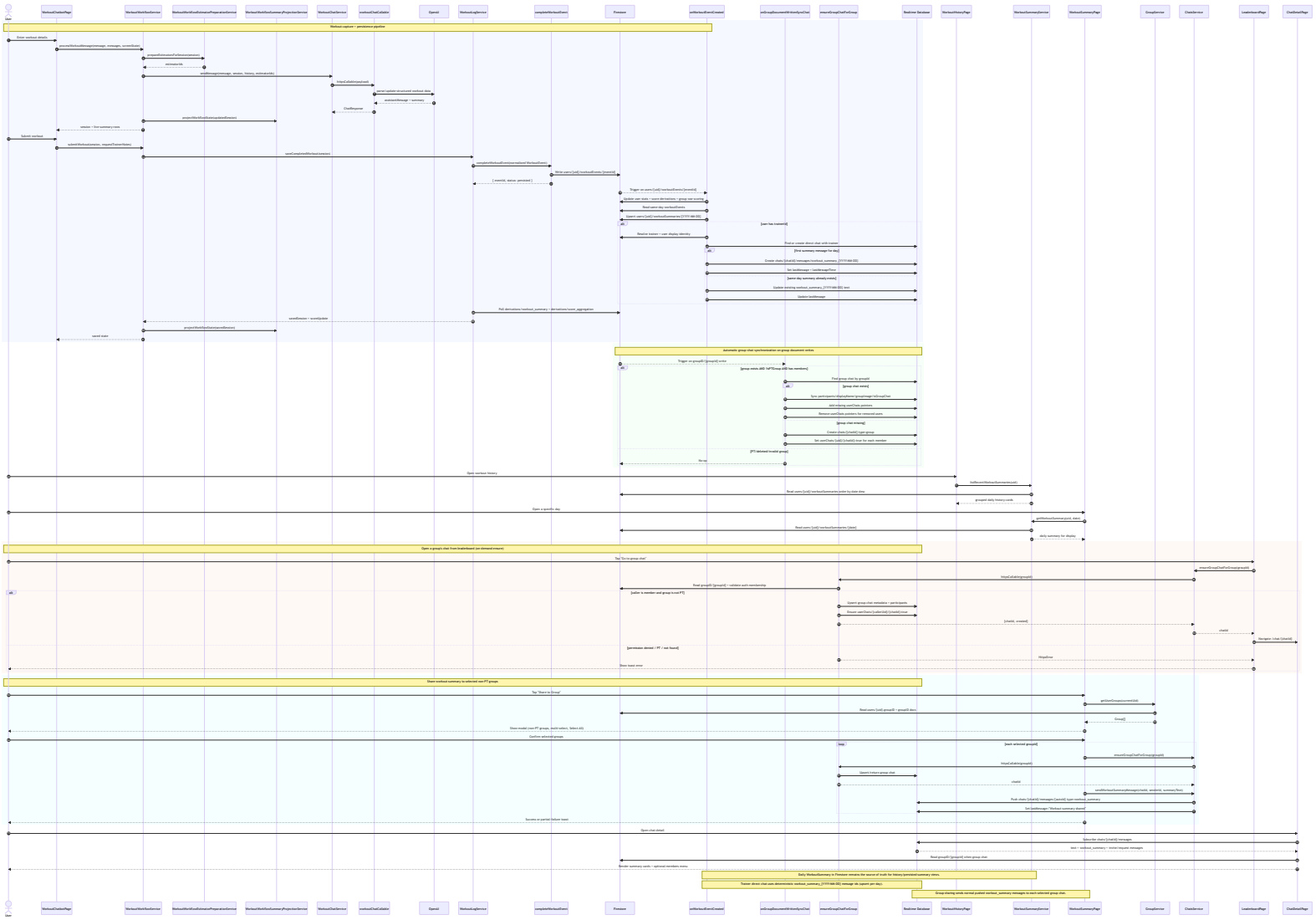
User
WorkoutCreationPage
WorkoutWorkflowService
WorkoutWorkflowEstimatorPreparationService
WorkoutWorkflowSummaryProjectionService
WorkoutChatService
workoutChatCallable
OpenAI
WorkoutLogService
completeWorkoutEvent
Firestore
setWorkoutEventCreated
onGroupDocumentWriteSyncChat
ensureGroupChatForGroup
Realtime Database
WorkoutHistoryPage
WorkoutSummaryService
WorkoutSummaryPage
GroupService
ChatService
LeaderboardPage
ChatDetailPage
Workout capture → persistence pipeline
Enter workout details
saveWorkoutMessage(message, messages, currentDraft)
prepareEstimatorPreparation(session)
estimate-lib.
sendMessage(message, session, history, estimateInfo)
bridgeCallable(payload)
server:update-structured workout data
assistantMessage + summary
ChatResponse
projectWorkoutUpdate(updatedSession)
session + live summary rows
Submit workout
submitWorkout(session, requestTrainerHome)
saveCompletedWorkout(session)
completeWorkoutEvent(normalized WorkoutEvent)
Write users/{uid}/workoutEvents/{eventId}
{ eventId, status: persisted }
Trigger on users/{uid}/workoutEvents/{eventId}
Update user stats + score derivations + group rate scores
Read same-day workoutEvents
Upsert users/{uid}/workoutSummaries/{YYYY-MM-DD}
alt
[user has trainerId]
Resolve trainer + user display identity
Find or create direct chat with trainer
alt
[first summary message for day]
Create chats/{chatId}/messages/workout_summary_{YYYY-MM-DD}
Set lastMessage + lastMessageTime
[same day summary already exists]
Update existing workout_summary_{YYYY-MM-DD} text
Update lastMessage
Poll derivations/workout_summary + derivations/user_aggregates
saveSession + scoreUpdate
projectWorkoutSummaryRow(saveSession)
saved state
Automatic group chat synchronization on group document writes
Trigger on groups/{{groupId}.write
alt
[group exists AND TeFTGroup AND has members]
Find group chat by groupId
[group chat exists]
alt
Sync participants/displayNames/groupImage/isGroupChat
Add missing userChats pointers
Remove userChats pointers for removed users
[group chat missing]
Create chats/{chatId}.type=group
Set userChats/{uid}/chatId.true for each member
[PT/I deleted/invalid group]
No-op
Open workout history
listRecentWorkoutSummaries(uid)
Read users/{uid}/workoutSummaries order by date desc
grouped daily history cards
Open a specific day
getWorkoutSummary(uid, date)
Read users/{uid}/workoutSummaries/{date}
daily summary for display
Open a group's chat from leaderboard (on-demand ensure)
Tap "Go to group chat"
ensureGroupChatForGroup(groupId)
bridgeCallable(groupId)
alt
Read groups/{{groupId} + validate auth membership
[caller is member and group is not PT]
Upsert group chat metadata + participants
Ensure user chats/{callerUid}/{chatId}.true
{chatId, created}
chatId
Navigate /chat/{chatId}
[permission denied / PT / not found]
HttpsError
Show toast error
Share workout summary to selected non-PT groups
Tap "Share to Group"
getUserGroups(currentUid)
Read users/{uid}.groupIDS + groupIDS docs
Group[]
Show modal (non-PT groups, multi-select), Select all()
Confirm selected groups
loop
[each selected groupId]
ensureGroupChatForGroup(groupId)
bridgeCallable(groupId)
Upsert /rooms group chat
chatId
sendWorkoutSummaryMessage(chatId, senderId, summaryText)
Push chats/{chatId}/messages/{autoId} type:workout_summary
Set lastMessage= "Workout summary shared"
Success or partial failure toast
Open chat detail
Subscribe chats/{chatId}/messages
text = workout_summary + invoke(request messages
Read groups/{{groupId} when group chat
Render summary cards + optional members notes
Daily WorkoutSummary in Firestore remains the source of truth for history (precisted summary rows).
Trainer direct chat uses deterministic workout_summary_{YYYY-MM-DD} message ids (upsert per day).
Group sharing sends normal pushed workout_summary messages to each selected group chat.